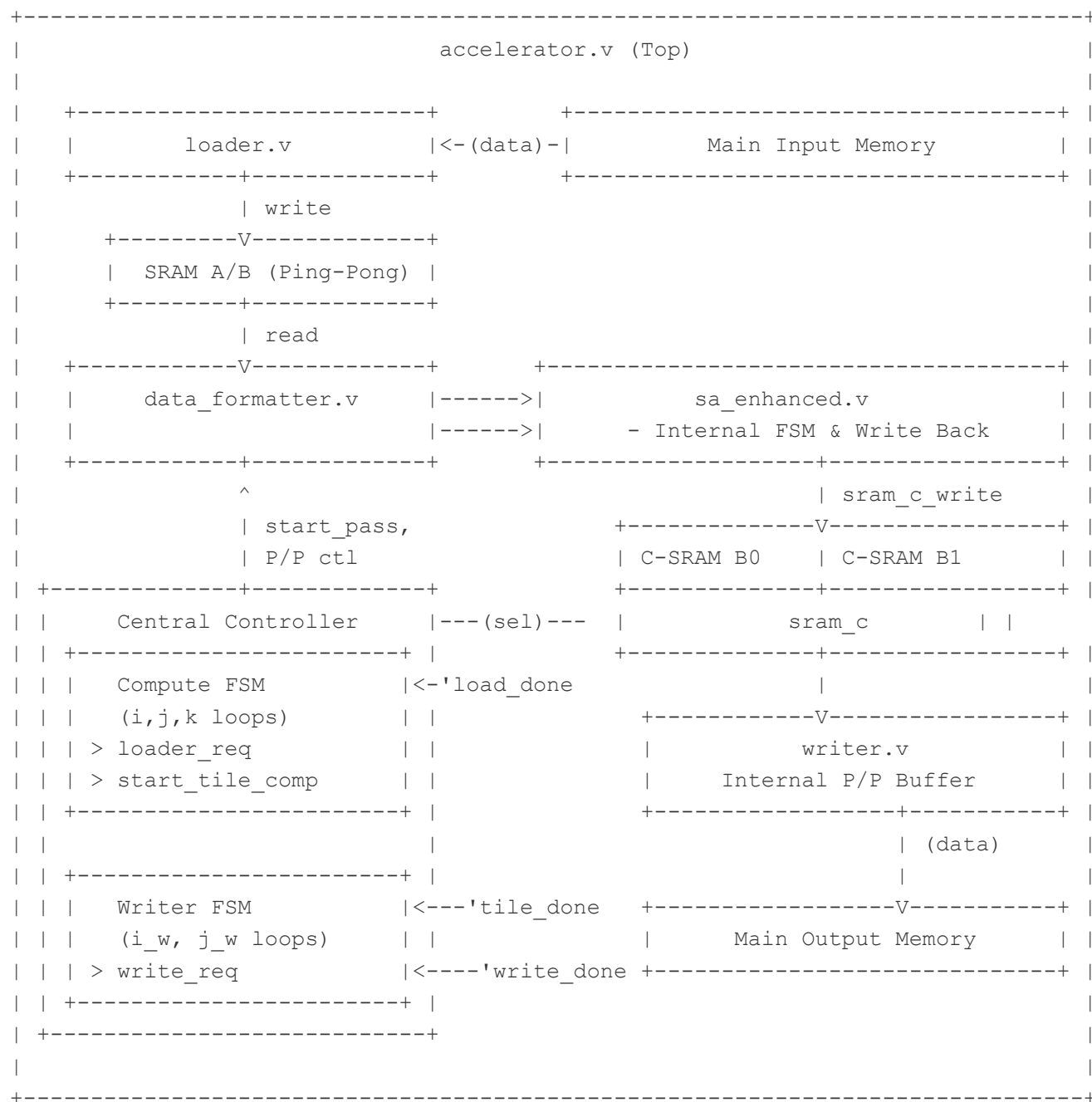


矩阵乘法加速器项目总结

0、 矩阵乘法加速器架构

这是一个用于执行通用矩阵乘法（GEMM）的高性能、可综合的硬件加速器。其核心设计思想是利用脉动阵列（Systolic Array）进行大规模并行计算，并通过深度流水线和双缓冲（Ping-Pong Buffering）机制来隐藏内存访问延迟，最大化计算单元的利用率。

一个架构示意图



测试说明:

仿真: `bash run.sh [matrix_dim]` 比如 `bash run.sh 512`。终端会100周期输出一次。512要226w周期左右

综合: `bash yosys_setup.sh` 然后 `bash synth.sh` 即可。综合结果在 `output_synth_V4` 文件夹中

一、模块功能简介与整体计算流程

本设计是一个高性能、可综合的通用矩阵乘法（GEMM）硬件加速器。它通过精细的模块化设计，将复杂的计算任务分解为加载、格式化、计算和写回等独立单元，并通过顶层控制器进行高效调度。

1. 各模块功能简介

- **tb_accelerator.v (测试平台)**: 模拟加速器的外部环境，包括一个单端口主存模型。它负责从 `input_mem.csv` 加载初始矩阵数据，通过 `comp_enb` 信号启动加速器，等待 `done` 信号，最后将计算结果从模拟主存中导出到 `result_mem.csv` 以供验证。
- **accelerator.v (顶层控制器)**: 加速器的“大脑”。通过两个并行的FSM（计算FSM和写回FSM）管理整体的 `i, j, k` 三层循环，调度所有子模块，并控制A/B/C三组SRAM的Ping-Pong缓冲切换，是实现宏观流水线的核心。
- **loader.v (数据加载器)**: 负责根据顶层控制器提供的瓦片索引(`i, k` 和 `k, j`)，计算主存地址，并从主存中读取A和B矩阵的瓦片数据，然后写入片上SRAM A/B。
- **sram_banked.v (A/B瓦片SRAM)**: 行为级建模的Banked SRAM，用于存储A和B矩阵瓦片。其核心特点是提供并行读端口，允许 `data_formatter` 在一个周期内读取一整行或一整列的数据，为数据歪斜提供高带宽支持。
- **data_formatter.v (数据格式化器)**: 负责从 `sram_banked` 中并行读取数据，并进行数据歪斜(Skewing)处理，将标准的矩阵数据流转换为脉动阵列所需的对角线波前 (diagonal wavefront) 数据流。
- **sa_enhanced.v (增强型脉动阵列)**: 加速器的计算核心。它实例化了 `SIZE x SIZE` 个PE单元，负责执行实际的乘加运算。内部FSM管理K维度的累加，并在计算完成后将结果写回SRAM C。
- **pe.v (处理单元)**: 脉动阵列的最基本单元，内含一个三级流水线的乘法累加器(MAC)。它执行 `C += A * B` 操作，并将A和B数据传递给相邻的PE。
- **sram_c.v (C瓦片SRAM)**: 一个标准的1读1写同步SRAM，用于缓存 `sa_enhanced` 计算出的C矩阵瓦片结果，作为计算和写回两个阶段的解耦缓冲。
- **writer.v (数据写回器)**: 负责从SRAM C中读取计算完成的C瓦片结果，将其打包成主存总线宽度（64位）的数据，并写回到主存的正确位置。

2. 整体计算流程模拟

启动: Testbench通过 `comp_enb` 信号启动 `accelerator`。

加载: `accelerator` 的计算FSM启动，向 `loader` 发出指令，加载第一个C瓦片（如 `C(0,0)`）所需的第一个K迭代数据（`A(0,0)` 和 `B(0,0)`）到SRAM A/B的一个缓冲（如Ping缓冲）中。

计算: 加载完成后，`data_formatter` 开始从Ping缓冲读取数据，进行歪斜处理后送入 `sa_enhanced`。
`sa_enhanced` 的PE阵列开始执行乘加运算。

K级流水: 在 `sa_enhanced` 计算 `k=0` 迭代的同时，`loader` 已经开始预取 `k=1` 迭代的数据（`A(0,1)` 和 `B(0,1)`）到SRAM A/B的另一个缓冲（Pong缓冲）中。当 `k=0` 计算完成，`k=1` 的数据也已准备就绪，SRAM A/B缓冲无缝切换，计算得以持续进行。此过程循环直到所有K次迭代完成。

瓦片级流水: 当 `C(0,0)` 瓦片的所有K次迭代计算完成，`sa_enhanced` 将最终结果写入 `sram_c`。几乎同时，`accelerator` 的写回FSM被触发，指示 `writer` 开始将 `sram_c` 中的 `C(0,0)` 数据写回主存。与此同时，`accelerator` 的计算FSM已经开始调度下一个瓦片（如 `C(0,1)`）的加载和计算流程。

完成: 整个过程以三级流水线的方式（加载、计算、写回）在不同瓦片粒度上重叠进行，直到所有C瓦片计算并写回完成，`accelerator` 拉高 `done` 信号，任务结束。

二、深度流水线设计：性能的精髓

本设计的核心优势在于其**跨越不同数据粒度的三级深度流水线**，它将加载、计算和写回操作在时间上最大程度地重叠，极大地隐藏了内存访问延迟，使得核心计算单元的利用率最大化。这与简单的“加载-计算-写回”串行方案有本质区别，后者会导致计算单元在加载和写回期间大量空闲。

1. K级流水线：计算与加载的无缝并行

这是加速器最重要的微架构流水线，旨在**完全隐藏DRAM访问延迟**。

- **机制**: 当脉动阵列(`sa_enhanced`)正在处理第 `k` 个数据切片时，`loader` 模块已经提前在为第 `k+1` 个数据切片从主存请求数据。
- **实现**:
 - **预取(Prefetching)**: `accelerator` 的计算FSM在启动 `data_formatter` 处理当前 (第 `k` 个) 数据后，会立即向 `loader` 发出针对下一个 (第 `k+1` 个) 数据切片的加载请求。
 - **SRAM A/B双缓冲(Ping-Pong Buffering)**: `loader` 将预取到的 `A(i, k+1)` 和 `B(k+1, j)` 瓦片写入当前空闲的SRAM A/B缓冲 (如Pong缓冲)，而 `sa_enhanced` 则从另一组存有当前数据的缓冲 (如Ping缓冲) 中读取。当 `k` 次迭代计算完成且 `k+1` 数据加载也完成后，FSM翻转SRAM A/B的选择位，实现计算和加载的无缝切换。

2. 瓦片级流水线：计算与写回的高度重叠

在更高的粒度上，当一个 `C_ij` 瓦片的计算与另一个 `C_{i,j-1}` 瓦片的写回也可以重叠。

- **机制**: 当一个C瓦片的计算 (所有k次迭代) 完成后，其写回主存的操作可以与下一个C瓦片的计算并行进行。
- **实现**:
 - **计算FSM与写回FSM的分离**: `accelerator.v` 内部设计了两个独立的FSM。**计算FSM**专注于 `i, j, k` 循环的计算调度，而**写回FSM**则独立地处理已完成瓦片的写回请求。
 - **解耦缓冲SRAM C**: 当 `sa_enhanced` 完成一个C瓦片的计算并将其写入 `sram_c` 后，它会通知顶层控制器。写回FSM接收到信号后，立即驱动 `writer` 模块从 `sram_c` 读取数据并写回主存。与此同时，计算FSM已经开始为下一个C瓦片调度 `loader` 和 `sa_enhanced`，两者互不干扰，实现了宏观上的流水。

3. 微观流水线优化：进一步压榨性能

除了宏观流水线，模块内部也充满了精细的流水线设计。

- **`sa_enhanced` 的提前写回**: 在处理最后一个K迭代时，`sa_enhanced` 无需等待整个瓦片所有PE都计算完毕。一旦某一行 (实际上使用的是半行，更节省 `sram_c`) 的结果准备就绪，它就立即开始将这部分结果写入SRAM C。这进一步缩短了计算到写回的延迟，使得瓦片级流水衔接更紧密。
- **`writer` 的内部流水线**: `writer` 模块内部也使用了一个Ping-Pong缓冲。它以 `SRAM_C_WRITE_WIDTH` (如256位) 的宽度从SRAM C读取数据 (生产者)，同时以 `MAIN_MEM_DATA_WIDTH_BITS` (64位) 的宽度向主存写入数据 (消费者)。通过**消费者驱动生产者 (Just-in-Time)** 的机制，只有当主存写入单元准备好消耗下一个64位字时，SRAM C的读取才可能被触发，这完美地解耦了SRAM C的高速读取和可能较慢的主存写入，避免了因主存反压造成的流水线停顿。

总结: 通过这套**宏观 (瓦片级)**、**中观 (K级)** 和**微观 (模块内部)** 三个层次的精密流水线设计，本加速器最大限度地减少了空闲周期，使得核心的脉动阵列计算单元能够持续满载运行，从而实现极高的计算吞吐率，已经接近纯加载 210w周期。但是继续优化延时毫无意义，因为IPE的Score更好😂。

SRAM资源配置

模块	用途	单缓冲容量	接口位宽	备注
sram_banked (A)	存储A矩阵瓦片	2 Kbits (16x16x8b)	写: 64-bit, 读: 128-bit	Ping-Pong双缓冲, Banked结构支持并行读
sram_banked (B)	存储B矩阵瓦片	2 Kbits (16x16x8b)	写: 64-bit, 读: 128-bit	Ping-Pong双缓冲, Banked结构支持并行读
sram_c	缓存C矩阵瓦片结果	8 Kbits (16x16x32b)	写/读: 256-bit	作为计算和写回阶段的解耦缓冲

三、性能、功耗与面积 (PPA) 分析

本节对加速器的性能、功耗和面积进行详细分析。分析基于 `MATRIX_SIZE=512` 和 `TILE_SIZE=16` 的配置，并根据 `Score = Power * Area^2 * Time` 的优化目标进行SRAM选型。

- 1. 性能 - 延迟 (Latency)
 - 仿真周期数: 2,262,210 cycles
 - 最大运行时钟频率: 根据时序分析，为满足时序收敛 (slack > 0)，设计可达到的最大频率为 **898 MHz**。
 - 最小运行时钟周期: $1 / 898 \text{ MHz} \approx 1.113 \text{ ns}$
 - 最终延迟: $2,262,210 \text{ cycles} \times 1.113 \text{ ns/cycle} \approx 2518 \mu\text{s}$
- 2. SRAM 资源分析与最终选型

为了在给定的评价标准下获得最优分数，我们对SRAM的选型进行了细致的面积和功耗权衡。

- **SRAM A/B 需求:** 每个缓冲需 2 Kbits 容量及 128-bit 读端口。
- **SRAM C 需求:** 需 8 Kbits 容量及 256-bit 读写端口。

经过对不同SRAM宏及其组合方式的全面评估，我们发现采用**单一类型的SRAM宏（宏#7）**作为所有片上存储的统一构建模块较优。

- 统一选型: 宏#7 (32x128, 4Kbits)
 - 单宏面积: **2034.22 μm^2**
 - 单宏功耗 (@898MHz): $310.31\mu\text{W (leakage)} + 16.98\mu\text{W/MHz} * 898\text{MHz} = 15551 \mu\text{W} = \mathbf{15.55 \text{ mW}}$
- 最终配置:
 - **SRAM A/B (双缓冲):** 2个缓冲/SRAM * 2个SRAM (A和B) * 1个宏#7/缓冲 = **4个 宏#7**。
 - **SRAM C:** 为构成256-bit接口，需并行使用 **2个 宏#7**。
 - **总计:** 共使用 4 (A/B) + 2 (C) = **6 个宏#7**。

- 3. 面积 (Area)
 - 逻辑单元面积: **326,145.26 μm^2**
 - SRAM 宏单元面积:
 - SRAM总面积: $6 \times 2034.22 \mu\text{m}^2 = \mathbf{12,205.32 \mu\text{m}^2}$
 - 加速器总面积:
 - 总面积 = 逻辑面积 + SRAM面积 = $326,145.26 + 12,205.32 = \mathbf{338,350.58 \mu\text{m}^2}$

- 4. 功耗 (Power)
 - 逻辑单元功耗: **154.80 mW (@ 898 MHz)**
 - SRAM 宏单元功耗 (基于活动因子估算):
 - SRAM A/B功耗 (4个宏, 50%活动因子): $4 \times 15.55 \text{ mW} \times 0.5 = 31.10 \text{ mW}$
 - SRAM C功耗 (2个宏, ~9%活动因子): $2 \times 15.55 \text{ mW} \times 0.09 = 2.80 \text{ mW}$

- SRAM总功耗: $31.10 + 2.80 = \mathbf{33.90\text{ mW}}$

• 加速器总功耗:

- 总功耗 = 逻辑功耗 + SRAM功耗 = $154.80\text{ mW} + 33.90\text{ mW} = \mathbf{188.70\text{ mW}}$

四、 最终PPA总结与优化得分

指标	值	备注
延迟 (Time)	2518 μs	512x512矩阵乘法 @ 898 MHz
面积 (Area)	338,350.58 μm^2	逻辑 (326,145.26) + SRAM (12,205.32)
功耗 (Power)	188.70 mW	逻辑 (154.80) + SRAM (33.90)

根据优化目标 $\text{Score} = \text{Power} * \text{Area}^2 * \text{Time}$ ，我们计算最终得分为：

• $\text{Score} = 188.70\text{ (mW)} \times (338,350.58\text{ (}\mu\text{m}^2))^2 \times 2518\text{ (}\mu\text{s)} =$

$$\mathbf{5.41 \times 10^{16}}$$

附注：关于 `sram_banked` 物理实现的说明

当前设计中，`sram_banked` 被建模为理想化的多端口存储器，这是一个简化处理。在物理实现上，为支持并行读取，它应由16个独立的单端口SRAM构成，同时loader应该有ping-pong缓冲机制，但这并不会显著影响整体性能。然而，SRAM宏库并没有8-bit参数，这可能导致需选用容量更大的宏，从而增加总面积和功耗。